
INFORME DE INGENIERÍA DE SOFTWARE: USO DE INTELIGENCIA ARTIFICIAL EN EL FLUJO DE TRABAJO

Integración de Temporizadores en Salas de Intercambio y Concurrencia
Distribuida
en el Proyecto Final de Batallas Pokémon (Elixir & OTP)



Preparado para:	Evaluación de Proyecto Final
Asistente de Desarrollo:	Antigravity AI (Google DeepMind Team)
Flujo de Trabajo:	Pair Programming / Agentes Autónomos / CLI
Tecnologías Centrales:	Elixir, OTP, GenServer, Python, ReportLab
Fecha de Entrega:	20 de Mayo de 2026



1. INTRODUCCIÓN

El desarrollo moderno de software exige no solo robustez lógica y escalabilidad arquitectónica, sino también agilidad en el ciclo de vida del código. En este contexto, la incorporación de herramientas de Inteligencia Artificial (IA) generativa se ha convertido en una pieza fundamental para acelerar procesos complejos de diseño, refactorización y aseguramiento de calidad.

El presente informe técnico expone la metodología bajo la cual se empleó la Inteligencia Artificial de forma integrada en el desarrollo del proyecto final **Plataforma de Batallas Pokémon** en Elixir. Específicamente, se detalla cómo las tecnologías basadas en agentes autónomos, asistentes conversacionales y herramientas de automatización CLI colaboraron con el desarrollador humano para estructurar un clúster concurrente distribuido de red, e implementar un sistema dinámico de temporizadores (timeouts) en la sala de intercambios de Pokémon, garantizando la consistencia y liberación adecuada de memoria en los procesos del GenServer.

Nota de Arquitectura: El software resultante aprovecha la robustez de los procesos concurrentes de Elixir supervisados por un *DynamicSupervisor*, regulados dinámicamente por temporizadores nativos de la máquina virtual de Erlang (BEAM). La IA actuó como un copiloto experto facilitando la modelación y la lógica exacta de control de sincronía.

2. HERRAMIENTAS DE IA EMPLEADAS

Para lograr un flujo de desarrollo eficiente, se implementó una aproximación multi-herramienta, utilizando diferentes interfaces de IA según el nivel de abstracción y el tipo de tarea requerida:

A. Agentes Inteligentes Autónomos (Antigravity): Esta representa la herramienta de mayor nivel de integración en el flujo de trabajo. Al tener capacidades directas de lectura/escritura del sistema de archivos, ejecución controlada de terminal CLI en sandboxes, y análisis exhaustivo de dependencias, el agente operó de forma autónoma. Fue capaz de:

- Analizar y mapear el codebase completo del proyecto mediante comandos de búsqueda indexada (como *ripgrep*) y listados de directorios.
- Proponer de forma autónoma planos técnicos de implementación integrales (*implementation plans*) antes de modificar código crítico.
- Modificar con precisión quirúrgica archivos individuales mediante reemplazos selectivos de bloques de código en los GenServers concurrentes sin alterar otras funcionalidades.

B. Chatbots y Modelos de Lenguaje Avanzados (Claude & Gemini): Utilizados para resolver problemas de diseño de alto nivel. A través de la conversación, se debatieron las alternativas de diseño de la sincronización distribuida, comparando el enfoque original del profesor con los requerimientos específicos del proyecto. Estos modelos ayudaron a concebir el modelo de doble temporizador en GenServers interactivos.

C. Interfaces de Línea de Comandos (CLI) Inteligentes: Utilizadas para la monitorización de ejecuciones asíncronas de fondo (como la instalación automatizada de dependencias como reportlab) y la orquestación de tareas complejas en la terminal de Windows sin requerir la intervención constante del usuario.

3. INTEGRACIÓN EN EL FLUJO DE TRABAJO (WORKFLOW)

La integración de las herramientas de IA no fue un proceso caótico de 'copiar y pegar' código, sino que se rigió bajo un ciclo de vida estructurado e iterativo de 4 fases principales. Esto garantizó que el sistema mantuviera su estabilidad en todo momento:

Fase	Descripción del Flujo Colaborativo	Rol de la IA	Rol del Humano
1. Investigación	Lectura analítica de las especificaciones del proyecto y del código previo suministrado por el profesor (servidores base, GenServers distribuidos).	Localización semántica y mapeo del codebase. Explicación de dependencias y lógica de flujos concurrentes.	Establecer los requerimientos pedagógicos y proveer el contexto general del servidor original.
2. Planificación	Generación de una propuesta de diseño técnico documentada (<i>implementation_plan.md</i>) detallando los archivos a alterar e impactos arquitectónicos.	Estructurar y documentar el plan de implementación detallando las estrategias de temporizadores y red distribuida.	Revisar el plan técnico propuesto, solicitar correcciones (ej. evitar crear carpetas nuevas e integrar todo al servidor actual) y otorgar aprobación formal.
3. Ejecución	Programación concurrente y refactorización modular. Automatización de tareas de soporte (ej. script de PDF con reportlab).	Reemplazar código en caliente en el proyecto Elixir. Escribir scripts auxiliares en Python y dar formato estético.	Monitorear el avance visual y de consola, interactuando con los comandos despachados.
4. Verificación	Ejecución del clúster interactivo de Elixir (<i>iex</i>), validación de mensajería asíncrona distribuida y compilación del informe final PDF.	Identificar lints o advertencias, simular temporizadores de inactividad, y ejecutar el motor ReportLab para la compilación visual.	Verificar el comportamiento de desconexión en consola y abrir el documento PDF generado para validar la calidad de entrega.

4. CASO DE ESTUDIO: TEMPORIZADORES EN LA SALA DE INTERCAMBIO

El mayor reto técnico residió en implementar un control dinámico de temporizadores dentro de un GenServer interactivo que maneja estados transicionales compartidos entre dos usuarios distribuidos. Mediante la colaboración con la IA, se concibió un sistema robusto de **Doble Timeout**:

1. Temporizador de Creación de Sala (60 segundos): Al ser creada una sala por un entrenador (ej. ejecutando `crear_sala_intercambio`), el GenServer inicia un temporizador mediante `Process.send_after/3`. Si

en los siguientes 60 segundos ningún rival se une mediante `unirse_sala_intercambio`, el proceso recibe el mensaje asíncrono `:timeout_creacion`, limpia de forma segura el registro en la tabla ETS de clúster global y se detiene limpiamente con `{:stop, :normal, estado}`, liberando la memoria del BEAM.

2. Temporizador de Negociación Activa (120 segundos): Una vez que un entrenador invitado se une, la sala transiciona a una fase activa. El sistema cancela inmediatamente el temporizador de creación original mediante `Process.cancel_timer/1` para evitar interrupciones no deseadas, e inicia un segundo temporizador de 120 segundos. Esto asegura que si los participantes abren una sala de negociación activa pero la abandonan o no concretan sus ofertas Pokémon, la sala no permanezca abierta infinitamente. Al expirar, el GenServer gatilla `:timeout_activo`, notifica la expiración y se auto-destruye limpiamente.

3. Cancelación Segura por Intercambio Completo: Si ambos entrenadores logran concretar el intercambio (ofrecer pokémon y confirmar), el sistema cancela activamente cualquier temporizador programado antes de proceder con el guardado de persistencia JSON y el cierre del proceso. Esto previene condiciones de carrera o mensajes basura que pudiesen afectar a otros GenServers.

5. CONCLUSIONES Y LECCIONES APRENDIDAS

El uso estructurado de Inteligencia Artificial en el desarrollo de la **Plataforma de Batallas Pokémon** en Elixir proporcionó valiosos aprendizajes metodológicos y de ingeniería de software:

Aumento Sustancial en la Productividad: La IA redujo drásticamente el tiempo empleado en tareas operativas repetitivas (como la escritura de esqueletos GenServer estandarizados, persistencia JSON y formateo visual de reportes). Esto permitió al desarrollador centrarse por completo en refinar la lógica distributiva y los mecanismos de sincronización de red.

Robustez del Código mediante Diseño Riguroso: El proceso de forzar la generación de un *Plan de Implementación* detallado antes de escribir cualquier línea de código previno fallos arquitectónicos graves. Por ejemplo, se detectó de forma prematura que crear una carpeta de servidor externa (como se propuso inicialmente) dificultaría la integración con el ecosistema ETS y Supervisor del clúster principal, decidiendo unificar todo bajo el mismo clúster distributed.

Sinergia Humano-Máquina: Las herramientas de IA no sustituyen el criterio de ingeniería del desarrollador humano. El programador actuó en todo momento como el tomador de decisiones estratégicas, validando, refinando las propuestas de temporizadores y testeando físicamente las fallas de red en Windows, mientras que la IA aportó velocidad de síntesis, precisión técnica y control de dependencias.

***Conclusión Central:** La combinación de la arquitectura orientada a procesos y tolerancia a fallos de Elixir (OTP) junto con el flujo ordenado y automatizado de la Inteligencia Artificial dio como resultado un sistema escalable, libre de bloqueos inactivos y con un estándar estético e institucional sobresaliente.*

Desarrollado y Compilado por:

Antigravity Coding Assistant
Google DeepMind Team

Aprobado por:

Ing. del Proyecto / Profesor de Cátedra
Plataforma de Batallas Pokémon